INSTITUTO FEDERAL DE SERGIPE CAMPUS LAGARTO
BACHARELADO EM SISTEMAS DE INFORMAÇÃO

Professor(a): Jose Solenir Lima Figueredo **Disciplina:** Algoritmo e Estrutura de Dados I

Monitor(a): José Gustavo Correia Nascimento **Data:** Maio/2026

Estudante: _____

LISTA DE EXERCÍCIOS DE REVISÃO

Questão 1: Auditoria de Lotes (Conceito)

Em uma fábrica de peças automotivas, os inspetores de qualidade registram a quantidade de peças defeituosas encontradas diariamente. Esses registros são armazenados em um vetor (array) de inteiros. Para automatizar o relatório semanal, você deve implementar um sistema que calcule o total de peças defeituosas utilizando **recursividade linear**.

Requisitos:

- Implemente um método recursivo chamado `somarDefeitos` que receba o array de registros e o tamanho do array (ou o índice atual) como parâmetros.
- O método deve retornar a soma total dos elementos do array.
- Identifique e implemente claramente o **caso base** e o **passo recursivo**.
- Crie uma classe `Main` para testar o seu método com o array `{2, 5, 1, 0, 4}`.

Pergunta Reflexiva: Explique, com suas palavras, o que acontece com a pilha de execução (*Call Stack*) do Java a cada nova chamada recursiva do método `somarDefeitos`. Como as variáveis locais e o estado de cada chamada são mantidos em memória até que o caso base seja atingido?

Questão 2: Validador de Códigos de Segurança (Prática)

Um sistema de cofre digital utiliza códigos de acesso simétricos para liberar a abertura das portas. Um código é considerado válido apenas se for um palíndromo perfeito (ou seja, lido da mesma forma da esquerda para a direita e da direita para a esquerda, ignorando espaços e diferenças entre maiúsculas e minúsculas).

Requisitos:

- Desenvolva um método recursivo `isPalindromo` que receba uma `String` (o código) e os índices de início e fim da análise.
- O método deve retornar um valor `boolean` indicando se o código é válido.
- Realize o tratamento para ignorar espaços em branco e letras maiúsculas/minúsculas da `String` original na classe principal antes de passar a informação para o método recursivo.
- A recursão deve comparar os caracteres das extremidades e se aproximar do centro a cada chamada (recursão de cauda/linear).
- Teste sua implementação com os códigos `"Ovo"`, `"Ame o poema"` e `"Senha123"`.

Pergunta Reflexiva: Em implementações recursivas como esta, a definição correta do caso base é vital. O que ocorreria nos bastidores (em termos de consumo de recursos e comportamento da JVM) se o caso base nunca fosse alcançado? Justifique.

Questão 3: Rastreador de Transações Financeiras (Desafio)

Um banco processa milhões de transações diárias. Para otimizar o sistema de auditoria e detecção de fraudes, os IDs das transações (números inteiros) são armazenados em um array estritamente ordenado de forma crescente. Sua missão é implementar um motor de busca de alto desempenho

para localizar rapidamente o índice de uma transação suspeita, seguindo o paradigma de divisão e conquista.

Requisitos:

- Implemente um método recursivo de **Busca Binária** chamado `buscarTransacao` que receba o array de transações, o ID desejado e os limites do sub-array (início e fim).
- Se a transação for encontrada, retorne o índice correspondente no array.
- Se a transação não existir, retorne `-1`.
- Demonstre o funcionamento na classe `Main` utilizando um array contendo pelo menos 10 IDs ordenados e realize buscas por um ID existente e por um inexistente.

Pergunta Reflexiva: Ao utilizar a estratégia de divisão e conquista da Busca Binária recursiva, o tamanho do problema cai pela metade a cada chamada. Como essa característica afeta a profundidade máxima da pilha de chamadas (*Call Stack*) em comparação com uma busca recursiva simples de ponta a ponta (busca linear) para o mesmo array? Explique a vantagem de desempenho com base nisso.

Questão 4: Controle de Vagas de Estacionamento (Conceito)

Uma empresa de mobilidade urbana opera um estacionamento inteligente com um número fixo de vagas numeradas (de 0 a $N-1$). O sistema mantém um **array primitivo de Strings** de tamanho fixo, onde cada posição armazena a placa do veículo estacionado ou `null` se a vaga estiver livre. Você deve implementar um painel de controle básico para esse estacionamento.

Requisitos:

- Declare um array de `String` com capacidade fixa de **8 vagas**.
- Implemente um método `estacionar(String[] vagas, int posicao, String placa)` que registre um veículo em uma posição específica. Se a posição já estiver ocupada, o método deve exibir um aviso e **não** sobrescrever a placa existente.
- Implemente um método `liberar(String[] vagas, int posicao)` que remova o veículo da posição informada (atribuindo `null`).
- Implemente um método `exibirPainel(String[] vagas)` que percorra todo o array e imprima o estado de cada vaga (livre ou ocupada com a respectiva placa).
- Na classe `Main`, simule a seguinte sequência: estacione 4 veículos em posições à sua escolha, tente estacionar um 5º veículo em uma posição já ocupada, libere uma vaga e exiba o painel final.

Pergunta Reflexiva: O array de vagas foi criado com tamanho fixo de 8 posições. Suponha que o estacionamento precise expandir para 12 vagas. Explique, do ponto de vista da alocação de memória contígua, por que **não é possível** simplesmente "aumentar" o array existente. Qual seria a estratégia necessária para acomodar as novas vagas e qual o custo computacional envolvido nessa operação?

Questão 5: Gerenciador de Playlist Musical (Prática)

Uma plataforma de streaming de música permite que o usuário crie playlists com um limite máximo de músicas. Internamente, a playlist é representada por um **array primitivo de Strings** de capacidade fixa e uma variável `tamanho` que controla quantas músicas foram efetivamente inseridas. Você deve construir um gerenciador completo que demonstre as operações de inserção e remoção e seus custos.

Requisitos:

- Crie uma classe `Playlist` que encapsule:
  - Um array `String[] musicas` com capacidade fixa de **7**.
  - Um atributo `int tamanho` (inicialmente 0) para rastrear a quantidade real de músicas.
- Implemente o método `adicionarNoFim(String musica)` — insere no final da playlist. Caso a playlist esteja cheia, exiba uma mensagem de erro.
- Implemente o método `inserirNaPosicao(int posicao, String musica)` — insere uma mú-

sica em uma posição específica, **deslocando (shift)** todos os elementos subsequentes uma posição para a direita. Valide se a posição é válida e se há espaço disponível.

- Implemente o método `removerDaPosicao(int posicao)` — remove a música na posição indicada, **deslocando (shift)** todos os elementos subsequentes uma posição para a esquerda para preencher o "buraco". Atribua `null` à última posição antes ocupada e decremente `tamanho`.
- Implemente o método `exibir()` que mostre apenas as músicas válidas (do índice 0 até `tamanho - 1`).
- Na classe `Main`, demonstre: adicione 5 músicas ao final, insira uma música na posição 2, remova a música na posição 0, e exiba a playlist após cada operação. Imprima **quantos deslocamentos (shifts)** foram necessários em cada inserção e remoção.

Pergunta Reflexiva: Ao inserir uma música na posição 0 de uma playlist que já contém N músicas, todos os N elementos precisam ser deslocados uma posição para a direita. Qual é a complexidade de tempo (Big-O) dessa operação no **pior caso**? Compare com a inserção no final. Por que essa diferença é uma limitação fundamental dos arrays primitivos?

Questão 6: Sistema de Senhas de Atendimento (Desafio)

Um hospital utiliza um sistema de fila de atendimento baseado em senhas numéricas. Devido a uma restrição do sistema legado, a fila é implementada com um **array primitivo de inteiros** de tamanho fixo. O sistema precisa lidar com situações do mundo real como: pacientes que desistem (saem da fila), pacientes prioritários que precisam ser encaixados em posições específicas, e a possibilidade de a fila lotar. Além disso, o administrador quer uma opção de **redimensionamento manual** caso a capacidade se esgote.

Requisitos:

- Crie uma classe `FilaDeSenhas` com:
  - Um array `int[] senhas` com capacidade inicial de **5**.
  - Um atributo `int quantidade` para controlar o número real de senhas na fila.
  - Um atributo `int proximaSenha` (iniciando em 1) para gerar senhas sequenciais automaticamente.
- Implemente `chamarProximaSenha()` — gera a próxima senha e a insere no **final** da fila. Se a fila estiver cheia, exiba "Fila lotada!" e **não** insira.
- Implemente `encaixarPrioritario(int posicao)` — gera uma nova senha e a insere na posição indicada, deslocando os demais. Valide se há espaço e se a posição é válida.
- Implemente `desistir(int posicao)` — remove a senha na posição informada, deslocando os elementos restantes para a esquerda.
- Implemente `redimensionar(int novaCapacidade)` — cria um **novo** array com a nova capacidade, copia todos os elementos existentes para ele e substitui o array interno. Trate o caso em que a nova capacidade é menor que a quantidade atual de elementos.
- Implemente `exibirFila()` para mostrar as senhas na ordem atual e a capacidade total.
- Na classe `Main`, simule o seguinte cenário:
  1. Chame 5 senhas normais (lotando a fila).
  2. Tente chamar uma 6ª senha (deve mostrar "Fila lotada!").
  3. Redimensione a fila para capacidade 10.
  4. Encaixe um prioritário na posição 0.
  5. Faça a senha da posição 3 desistir.
  6. Exiba a fila final.

Pergunta Reflexiva: O método `redimensionar` cria um novo array e copia todos os elementos um a um. Sabendo que arrays ocupam **blocos contíguos de memória**, explique por que o Java (e a maioria das linguagens) não pode simplesmente "esticar" o array original na memória. O que acontece

com o array antigo após a cópia? Relacione sua resposta com o conceito de *Garbage Collection* da JVM.

Questão 7: Registro de Check-ins em Museu (Conceito)

Um museu interativo recém-inaugurado quer digitalizar o registro de visitantes. A cada dia, um número completamente imprevisível de pessoas pode aparecer: desde 5 em uma segunda-feira chuvosa até 300 em um feriado nacional. Usar um array de tamanho fixo seria impraticável, pois o museu não sabe antecipadamente quantos visitantes receberá. Sua tarefa é utilizar um `ArrayList<String>` para criar um sistema flexível de check-in.

Requisitos:

- Crie um `ArrayList<String>` chamado `visitantes`.
- Simule a chegada de **8 visitantes** ao museu, adicionando seus nomes à lista com o método `add()`.
- Após todos os check-ins, exiba o **total de visitantes** registrados utilizando `size()`.
- Exiba o **nome do 3º visitante** (índice 2) utilizando o método `get()`.
- Verifique se um visitante específico (à sua escolha) está na lista usando `contains()`.
- Exiba a lista completa de visitantes utilizando um laço `for-each`.

Pergunta Reflexiva: Um `ArrayList` internamente utiliza um array primitivo como estrutura de armazenamento. Sabendo que o `ArrayList` possui uma **capacidade inicial padrão de 10 posições** no Java, o que acontece nos bastidores da JVM quando você insere o 11º visitante? Descreva o processo de **redimensionamento automático** (criação de novo array, cópia de elementos, fator de crescimento) e explique por que, apesar do custo pontual dessa operação, o tempo **amortizado** de inserção continua sendo $O(1)$.

Questão 8: Gerenciador de Encomendas de Transportadora (Prática)

Uma transportadora regional precisa de um sistema para gerenciar a lista de encomendas pendentes do dia. O volume de entregas varia drasticamente: em dias comuns são 20 pacotes, mas em datas promocionais (Black Friday, Natal) podem chegar mais de 500. O sistema deve permitir adicionar encomendas conforme chegam, consultar encomendas por posição, localizar uma encomenda pelo código de rastreio e remover encomendas que forem despachadas.

Requisitos:

- Crie uma classe `GerenciadorEncomendas` que encapsule:
  - Um `ArrayList<String>` chamado `encomendas` para armazenar os códigos de rastreio.
- Implemente o método `registrar(String codigoRastreio)` — adiciona uma nova encomenda ao final da lista usando `add()`. Antes de inserir, verifique com `contains()` se o código já existe (não permita duplicatas).
- Implemente o método `inserirUrgente(int posicao, String codigoRastreio)` — insere uma encomenda prioritária em uma posição específica usando `add(int index, E element)`. Valide o índice com tratamento de exceção (`IndexOutOfBoundsException`).
- Implemente o método `consultar(int posicao)` — retorna o código de rastreio na posição indicada usando `get()`. Trate a exceção `IndexOutOfBoundsException`.
- Implemente o método `localizar(String codigoRastreio)` — retorna a posição (índice) da encomenda na fila usando `indexOf()`. Se não encontrada, retorne `-1`.
- Implemente o método `despachar(String codigoRastreio)` — remove a encomenda da lista usando `remove(Object)`. Exiba uma mensagem de sucesso ou de "encomenda não encontrada".
- Implemente o método `exibirPainel()` — lista todas as encomendas pendentes com seus índices e exiba a quantidade total via `size()`.
- Na classe `Main`, simule o seguinte cenário:

1. Registre 6 encomendas com códigos fictícios (ex: "BR123456789").
2. Tente registrar uma encomenda com código duplicado (deve ser rejeitada).
3. Insira uma encomenda urgente na posição 0.
4. Consulte a encomenda na posição 3.
5. Localize o índice de uma encomenda pelo seu código.
6. Despache 2 encomendas (removendo-as da lista).
7. Exiba o painel final.

Pergunta Reflexiva: O método `add(int index, E element)` do `ArrayList` permite inserir em qualquer posição válida. Internamente, o `ArrayList` usa o método nativo `System.arraycopy()` para deslocar os elementos subsequentes. Compare o custo dessa operação com a inserção que você implementou manualmente com `for` no Tema 2 (Array Primitivo). Existe diferença de complexidade Big-O? Qual a vantagem real que o `ArrayList` oferece em relação ao array primitivo, considerando o redimensionamento?

Questão 9: Filtro de Alertas em Sistema de Monitoramento IoT (Desafio)

Uma fábrica possui sensores IoT espalhados em diversas máquinas. Cada sensor gera alertas com um nível de severidade: `"INFO"`, `"WARNING"` ou `"CRITICAL"`. Ao final de cada turno, o operador precisa **filtrar** a lista de alertas removendo todos os alertas de nível `"INFO"` (considerados irrelevantes) para gerar um relatório apenas com os alertas que exigem atenção.

O grande desafio é: como remover elementos de um `ArrayList` **durante** uma iteração sem causar erros ou pular elementos?

Requisitos:

- Crie uma classe `Alerta` com os atributos:
  - `String sensor` — identificação do sensor (ex: "Sensor-T01").
  - `String nivel` — nível de severidade (`"INFO"`, `"WARNING"` ou `"CRITICAL"`).
  - `String mensagem` — descrição do alerta.
- Crie uma classe `PainelDeAlertas` que encapsule um `ArrayList<Alerta>` e implemente:
  - `registrar(Alerta alerta)` — adiciona um alerta à lista.
  - `exibirTodos()` — exibe todos os alertas com seus dados.
- Na classe `Main`, popule o painel com pelo menos **10 alertas** (misturando os três níveis de severidade).
- Implemente **três estratégias distintas de remoção** para remover os alertas `"INFO"`, cada uma em uma cópia separada da lista original:
  - **Estratégia A (INCORRETA):** Use um `for` tradicional com índice (`for (int i = 0; i < lista.size(); i++)`) e `remove(i)`. Observe como elementos são pulados.
  - **Estratégia B (CORRETA):** Utilize um `Iterator<Alerta>` explícito com os métodos `hasNext()`, `next()` e `remove()` para remoção segura.
  - **Estratégia C (MODERNA):** Utilize o método `removeIf()` com expressão lambda para remover alertas de forma concisa e segura.
- Após cada estratégia, exiba os alertas remanescentes e compare os resultados.

Pergunta Reflexiva: Na Estratégia A, ao remover o elemento no índice `i`, todos os elementos subsequentes são deslocados uma posição para a esquerda pelo `ArrayList`. Porém, a variável `i` é incrementada normalmente pelo `for`. Explique, passo a passo com um exemplo de 4 elementos, por que isso faz com que o elemento que estava imediatamente após o removido seja "pulado" pela iteração. Em seguida, explique como a abordagem de percorrer a lista **do final para o início** (`for`

`(int i = lista.size()-1; i >= 0; i-)`) resolveria o problema sem usar `Iterator`. Por que essa técnica funciona?

Questão 10: Laboratório de Análises Clínicas (Conceito)

Um laboratório de análises clínicas armazena amostras biológicas em tubos de ensaio. Cada tubo pode conter materiais de tipos diferentes: uma amostra de sangue é representada por uma `String` (o tipo sanguíneo, ex: "O+"), uma contagem de leucócitos por um `Integer` e uma taxa de glicose por um `Double`. O sistema legado do laboratório utilizava um **array de** `Object` para guardar qualquer tipo de amostra, exigindo *castings* explícitos e arriscando erros em tempo de execução (`ClassCastException`).

Sua missão é demonstrar o **antes e depois**: primeiro, reproduzir os problemas do código sem Generics; depois, criar uma classe genérica `Tubo<T>` que resolva esses problemas com segurança de tipos em tempo de compilação.

Requisitos:

- **Parte A — Sem Generics (código problemático):**
  - Crie um array `Object[] amostras` de tamanho 3.
  - Armazene uma `String` (tipo sanguíneo), um `Integer` (leucócitos) e um `Double` (glicose) no array.
  - Recupere o primeiro elemento e faça um *casting* correto para `String`. Exiba o resultado.
  - Agora, **propositalmente**, tente fazer um *casting* do segundo elemento (`Integer`) para `String`. Capture a exceção `ClassCastException` com `try-catch` e exiba uma mensagem de erro explicativa.
- **Parte B — Com Generics (solução segura):**
  - Crie uma classe genérica `Tubo<T>` com:
    - Um atributo privado `T amostra`.
    - Um método `void inserir(T amostra)` para armazenar a amostra.
    - Um método `T retirar()` que retorna a amostra armazenada.
  - Na classe `Main`, crie três tubos tipados: `Tubo<String>`, `Tubo<Integer>` e `Tubo<Double>`.
  - Insira e retire amostras de cada tubo **sem** necessidade de *casting*. Exiba os resultados.
  - Demonstre que tentar inserir um tipo incompatível (ex: inserir um `Integer` em um `Tubo<String>`) gera um **erro de compilação**, e não de execução. Comente essa linha no código e explique com um comentário.

Pergunta Reflexiva: Você observou que, com Generics, o compilador Java impede a inserção de tipos incompatíveis **antes mesmo** de o programa ser executado. Porém, nos bastidores, o Java aplica um mecanismo chamado **Type Erasure**: durante a compilação, todas as informações de tipo genérico são removidas, e `T` é substituído por `Object` no bytecode final. Sabendo disso, explique: se o bytecode compilado de `Tubo<String>` e `Tubo<Integer>` é **idêntico** (ambos usam `Object` internamente), *como* o compilador consegue rejeitar o código incorreto antes da compilação? Qual é a vantagem prática dessa abordagem para o desenvolvedor?

Questão 11: Cofre Digital Parametrizado (Prática)

Uma empresa de segurança patrimonial oferece cofres digitais inteligentes para seus clientes. Cada cofre pode guardar **um único tipo** de item valioso: alguns clientes guardam documentos (representados como `String`), outros guardam quantias em dinheiro (`Double`) e outros guardam códigos de acesso (`Integer`). O sistema deve ser parametrizado com Generics para que cada cofre tenha segurança de tipos: um cofre de documentos não pode acidentalmente receber uma quantia monetária.

Requisitos:

- Crie uma classe genérica `Cofre<T>` que encapsule:

- Um `ArrayList<T>` privado chamado `itens` para armazenar os objetos guardados.
- Um atributo `String dono` que identifica o proprietário do cofre.
- Um atributo `int capacidadeMaxima` que define o limite de itens do cofre.

- Implemente os seguintes métodos:
  - `boolean guardar(T item)` — adiciona um item. Retorna `false` se cheio. Não permita duplicatas (`contains()`).
  - `T recuperar(int posicao)` — retorna o item na posição informada. Trate *IndexOutOfBoundsException*.
  - `boolean contem(T item)` — verifica se um item está no cofre via `contains()`.
  - `T remover(int posicao)` — remove e retorna o item na posição indicada. Trate as exceções.
  - `void listarConteudo()` — exibe os itens, o dono e as vagas disponíveis.
- Na classe `Main`, crie e demonstre o funcionamento de **três cofres** distintos:
  1. `Cofre<String>` do cliente "Alice" (capacidade 5): guarde 4 documentos, tente guardar um duplicado (deve ser rejeitado), recupere o documento na posição 2 e liste o conteúdo.
  2. `Cofre<Double>` do cliente "Bruno" (capacidade 3): guarde 3 quantias, tente guardar uma 4ª (deve exibir "Cofre lotado!"), e remova a quantia na posição 1.
  3. `Cofre<Integer>` do cliente "Carla" (capacidade 4): guarde 2 códigos de acesso, verifique se um código específico está no cofre usando `contem()`.

Pergunta Reflexiva: Ao compilar o seu programa, o Java aplica **Type Erasure** e substitui o parâmetro `T` da classe `Cofre<T>` por `Object`. Isso significa que, no bytecode final, **só existe uma única classe** `Cofre` — e não três versões separadas (`Cofre_String`, `Cofre_Double`, `Cofre_Integer`) como seria em linguagens como C++ com templates. Explique por que essa decisão de design da linguagem Java (ter uma única classe compartilhada no bytecode) impede o uso de certas operações em tempo de execução, como `new T()` ou `instanceof T`. Dica: se `T` vira `Object` no bytecode, o que o programa "saberia" sobre `T` durante a execução?

Questão 12: Inventário de Loja Virtual com Limites de Tipo (Desafio)

Uma loja virtual vende diferentes categorias de produtos: eletrônicos, livros e roupas. Todos os produtos compartilham atributos em comum (nome, preço), mas cada categoria possui dados específicos. O sistema de inventário precisa de uma estrutura genérica que funcione **apenas com subtipos de** `Produto` — rejeitando qualquer outro tipo (como `String` ou `Integer`) em tempo de compilação. Além disso, a equipe de relatórios precisa de um método utilitário que consiga calcular o valor total de **qualquer lista de produtos**, independentemente da categoria, usando **Wildcards**.

Requisitos:

- Crie uma classe abstrata `Produto` com:
  - Atributos protegidos: `String nome` e `double preco`.
  - Construtor parametrizado e métodos `getNome()` e `getPreco()`.
  - Um método abstrato `String descricaoDetalhada()` que cada subcategoria deve implementar.
  - Sobrescreva `toString()` para exibir nome e preço formatado.
- Crie **três subclasses concretas** de `Produto`:
  - `Eletronico` — com atributo adicional `int garantiaMeses`.
  - `Livro` — com atributo adicional `String autor`.
  - `Roupa` — com atributo adicional `String tamanho`.
- Crie uma classe genérica `Inventario<T extends Produto>` que encapsule:

- Um `ArrayList<T>` privado chamado `estoque`.
- `void adicionar(T produto)` — insere um produto no estoque.
- `T buscarPorNome(String nome)` — percorre a lista e retorna o primeiro produto cujo nome corresponda (ignore maiúsculas/minúsculas). Retorne `null` se não encontrado.
- `void listar()` — exibe todos os produtos com sua `descricaoDetalhada()`.
- `int quantidade()` — retorna o total de produtos no estoque.

- Crie uma classe utilitária `RelatorioUtil` com um método **estático** que use **Wildcards**:
  - `static double calcularValorTotal (ArrayList<? extends Produto> lista)` — retorna a soma dos preços.
- Na classe `Main`, demonstre:
  1. Crie um `Inventario<Eletronico>` e um `Inventario<Livro>`.
  2. Utilize `RelatorioUtil.calcularValorTotal()` para exibir o valor total de cada inventário — demonstrando que o método aceita os diferentes tipos de *ArrayList* via Wildcard.
  3. Comente o que ocorre ao tentar criar um `Inventario<String>`.

Pergunta Reflexiva: Por que `ArrayList<Eletronico>` não é subtipo de `ArrayList<Produto>`? Como o Wildcard resolve essa invariância? No *Type Erasure*, o `T` de `<T extends Produto>` vira `Object` ou `Produto`?

Questão 13: Correia Transportadora de Fábrica (Conceito)

Uma fábrica de embalagens possui uma correia transportadora onde as caixas são processadas uma a uma, na ordem de chegada. Cada caixa carrega um código de identificação (`String`) e está fisicamente ligada à próxima caixa da linha por um gancho mecânico. Diferentemente de uma esteira com slots fixos (análoga a um array), a correia pode receber um número ilimitado de caixas pois não exige posições pré-alocadas — basta engatar uma nova caixa ao final da sequência.

Sua tarefa é modelar essa correia transportadora utilizando uma **Lista Simplesmente Encadeada** (Singly Linked List), implementando do zero as classes `No` (nó) e `CorreiaTransportadora` (lista).

Requisitos:

- Crie uma classe `No` (representando uma caixa na correia) com:
  - Um atributo `String codigo` — o código de identificação da caixa.
  - Um atributo `No proximo` — a referência (ponteiro) para a próxima caixa na correia. Inicialmente `null`.
  - Um construtor que receba o código e inicialize `proximo` como `null`.
- Crie uma classe `CorreiaTransportadora` com:
  - Um atributo `No cabeca` — ponteiro para a primeira caixa da correia. Inicialmente `null` (correia vazia).
  - Um atributo `int tamanho` — quantidade de caixas na correia. Inicialmente 0.
- Implemente os seguintes métodos:
  - `void adicionarNoInicio(String codigo)` — cria uma nova caixa e a coloca no **início** da correia (a nova caixa se torna a nova cabeça, e seu `proximo` aponta para a antiga cabeça).
  - `void adicionarNoFim(String codigo)` — cria uma nova caixa e a coloca no **final** da correia. Para isso, percorra a lista até encontrar o último nó (aquele cujo `proximo` é `null`) e enganche a nova caixa ali. Trate o caso da correia vazia.
  - `void exibirCorreia()` — percorre a correia do início ao fim, imprimindo cada código no formato `[CX-001] -> [CX-002] -> ... -> null`.
- Na classe `Main`, simule:
  1. Adicione 3 caixas no final da correia ("CX-101", "CX-102", "CX-103").

2. Exiba a correia.
3. Adicione 1 caixa no início ("CX-URGENTE").
4. Exiba a correia novamente e imprima o tamanho.

Pergunta Reflexiva: Observe que, para adicionar uma caixa no **início** da correia, bastou atualizar dois ponteiros (operação $O(1)$), enquanto para adicionar no **final** foi necessário percorrer toda a lista do início até o último nó (operação $O(n)$). Compare essa situação com a de um array primitivo, onde a inserção no final é $O(1)$ (se houver espaço) mas a inserção no início exige deslocar todos os elementos. Qual estrutura é mais eficiente para inserções frequentes no **início** dos dados? Justifique com base no conceito de alocação contígua versus não-contígua em memória.

Questão 14: Gerenciador de Processos de Sistema Operacional (Prática)

Um sistema operacional simplificado mantém uma lista de processos em execução. Cada processo possui um `int pid` (identificador único) e uma `String nome` (ex: "navegador", "editor"). Os processos são gerenciados em uma **Lista Simplesmente Encadeada**, permitindo que o S.O. inicie (insira) e finalize (remova) processos dinamicamente sem se preocupar com capacidade fixa.

Requisitos:

- Crie uma classe `Processo` com os atributos `int pid` e `String nome`, além de um construtor parametrizado e um método `toString()`.
- Crie uma classe `No` com:
  - Um atributo `Processo dado` — o processo armazenado no nó.
  - Um atributo `No proximo` — referência para o próximo nó.
- Crie uma classe `ListaDeProcessos` com o ponteiro `No cabeca` e `int tamanho`, e implemente:
  - `void iniciarProcesso(Processo p)` — insere o processo no **início** da lista (simulando prioridade ao processo mais recente). Complexidade: $O(1)$.
  - `void iniciarProcessoNoFim(Processo p)` — insere o processo no **final** da lista. Complexidade: $O(n)$.
  - `Processo finalizarPrimeiro()` — remove e retorna o processo que está na **cabeça** da lista (o mais recente). Trate o caso de lista vazia exibindo um aviso. Complexidade: $O(1)$.
  - `void exibirProcessos()` — percorre a lista e imprime todos os processos ativos no formato encadeado.
  - `int contarProcessos()` — retorna o tamanho da lista.
- Na classe `Main`, simule o seguinte cenário:
  1. Inicie 3 processos no início: (pid=1, "kernel"), (pid=2, "navegador"), (pid=3, "editor").
  2. Inicie 1 processo no final: (pid=4, "backup").
  3. Exiba todos os processos ativos.
  4. Finalize o primeiro processo (cabeça). Exiba qual processo foi finalizado.
  5. Exiba a lista atualizada e a contagem de processos.

Pergunta Reflexiva: Na operação `iniciarProcessoNoFim`, você precisou percorrer **todos** os nós da lista para encontrar o último. Em uma lista com 1 milhão de processos, essa travessia custaria $O(n)$ acessos à memória. Qual modificação estrutural na classe `ListaDeProcessos` eliminaria esse custo, tornando a inserção no final também $O(1)$? Descreva essa modificação e explique quais métodos precisariam ser ajustados para manter a consistência do novo ponteiro.

Questão 15: Central de Ocorrências Policiais (Desafio)

Uma delegacia de polícia registra boletins de ocorrência (B.O.) em um sistema legado baseado em uma **Lista Simplesmente Encadeada**. Cada B.O. possui um `int numero` (número do boletim, único), uma `String tipo` (ex: "Furto", "Acidente", "Agressão") e uma `String descricao` (breve relato). O sistema precisa suportar operações de busca e remoção de B.O.s no **meio** da lista — situações que exigem cuidado redobrado para evitar `NullPointerException` durante a travessia dos ponteiros.

Requisitos:

- Crie uma classe `BoletimOcorrencia` com os atributos `int numero`, `String tipo` e `String descricao`, além de construtor parametrizado e `toString()`.
- Crie uma classe `No` com atributo `BoletimOcorrencia dado` e `No proximo`.
- Crie uma classe `CentralOcorrencias` com `No cabeca`, `int tamanho`, e implemente:
  - `void registrar(BoletimOcorrencia bo)` — insere o B.O. no **final** da lista.
  - `BoletimOcorrencia buscarPorNumero(int numero)` — percorre a lista buscando o B.O. com o número informado. Retorne o objeto encontrado ou `null` se não existir. **Cuidado:** verifique se o nó atual não é `null` antes de acessar seus atributos para evitar `NullPointerException`.
  - `boolean removerPorNumero(int numero)` — remove o B.O. com o número informado, onde quer que ele esteja na lista. Para isso, utilize a técnica do **ponteiro anterior** (`anterior`): mantenha duas referências durante a travessia — uma para o nó atual e outra para o nó anterior. Ao encontrar o B.O. desejado, faça `anterior.proximo = atual.proximo` para "pular" o nó removido. Trate os casos especiais:
    - Remoção quando o B.O. está na **cabeça** da lista.
    - Remoção quando o B.O. **não é encontrado**.
    - Lista vazia.

    Retorne `true` se removido com sucesso, `false` caso contrário.
  - `void listarOcorrencias()` — exibe todos os B.O.s registrados com seus dados completos.
  - `int totalOcorrencias()` — retorna a quantidade de B.O.s na central.
- Na classe `Main`, simule o seguinte cenário:
  1. Registre 5 boletins de ocorrência com dados fictícios (números 1001 a 1005).
  2. Liste todas as ocorrências.
  3. Busque o B.O. número 1003 e exiba seus dados.
  4. Busque um B.O. inexistente (número 9999) e trate o retorno `null`.
  5. Remova o B.O. 1001 (cabeça da lista). Exiba a lista atualizada.
  6. Remova o B.O. 1003 (meio da lista). Exiba a lista atualizada.
  7. Remova o B.O. 1005 (final da lista). Exiba a lista atualizada e o total.
  8. Tente remover o B.O. 9999 (inexistente) e exiba a mensagem de falha.

Pergunta Reflexiva: Para remover um nó no **meio** de uma Lista Simplesmente Encadeada, foi necessário primeiro **encontrá-lo** (percorrendo a lista) e depois atualizar o ponteiro `proximo` do nó anterior. Essa operação de busca custa $O(n)$ no pior caso. Compare isso com um `ArrayList`, onde o acesso por índice é $O(1)$ (acesso direto). Por que uma lista encadeada **não** permite acesso direto por índice? Relacione sua explicação com a diferença entre alocação **contígua** de memória (os elementos ficam lado a lado, permitindo cálculo de endereço por aritmética) e alocação **não-contígua** (cada nó pode estar em qualquer lugar da memória, conectado apenas por referências). Em que situações essa "desvantagem" da lista encadeada é compensada por suas vantagens?

Questão 16: Histórico de Navegação de Internet (Conceito)

Os navegadores modernos utilizam uma estrutura de **Lista Duplamente Encadeada** para gerenciar o histórico de abas. Isso permite que o usuário navegue para frente e para trás de forma eficiente. Diferente da lista simples, cada nó aqui possui uma referência para o nó anterior (`prev`) e para o próximo (`next`). Para simplificar a lógica de borda (lista vazia ou com um elemento), utilizaremos a estratégia de **Nós Sentinelas** (`header` e `trailer`).

Requisitos:

- Crie uma classe `NoDuplo` com:
  - `String url` — o endereço do site visitado.
  - `NoDuplo anterior` e `NoDuplo proximo`.
  - Construtor que receba a URL e as referências para os nós vizinhos.
- Crie uma classe `HistoricoBrowser` que utilize sentinelas:
  - Atributos privados `header` e `trailer` (nós que não guardam dados).
  - No construtor, ligue `header.proximo = trailer` e `trailer.anterior = header`.
- Implemente os métodos:
  - `void visitar(String url)` — insere um novo site no **final** da lista (antes do `trailer`).
  - `void exibirOrdemDireta()` — percorre do `header` para o `trailer`.
  - `void exibirOrdemReversa()` — percorre do `trailer` para o `header`, demonstrando a navegação bidirecional.
- Na classe `Main`, simule a visita aos sites: "google.com", "github.com" e "stackoverflow.com". Exiba o histórico nas duas ordens.

Pergunta Reflexiva: Em uma Lista Duplamente Encadeada, cada nó armazena uma referência adicional (`prev`), o que aumenta o consumo de memória RAM em comparação com uma Lista Simples. Explique qual é a vantagem algorítmica fundamental desse "custo extra"ao realizar a operação de **remoção do último elemento** da lista. Por que na lista simples essa operação é $O(n)$ e na dupla ela pode ser $O(1)$?

Questão 17: Sistema Undo/Redo de Editor de Texto (Prática)

Um editor de texto armazena as ações do usuário (digitar, apagar, formatar) em uma Lista Duplamente Encadeada. Um ponteiro chamado `cursorAtual` indica em qual estado o editor se encontra. Ao clicar em "Desfazer"(Undo), o cursor move-se para o nó anterior. Ao clicar em "Refazer"(Redo), move-se para o próximo. Se o usuário realizar uma **nova ação** estando no meio do histórico, todos os estados à frente do cursor devem ser descartados.

Requisitos:

- Crie uma classe `EditorUndoRedo` que gerencie a lista de strings (ações).
- Mantenha uma referência `NoDuplo cursorAtual`.
- Implemente o método `novaAcao(String acao)`:
  - Adicione a ação após o `cursorAtual`.
  - **Importante:** Se houver nós após o cursor, eles devem ser desconectados da lista (perdendo a referência para o Redo).
  - Atualize o `cursorAtual` para este novo nó.
- Implemente `undo()` e `redo()`, movendo o cursor e exibindo a ação atual. Trate os limites (início e fim da lista) para evitar `NullPointerException`.
- Na classe `Main`, simule: Digitar "Olá", Digitar "Mundo", Undo, Digitar "Pessoal", tentar Redo (deve estar desabilitado/vazio).

Pergunta Reflexiva: O descarte dos nós à frente do cursor ao realizar uma nova ação após um Undo é essencial para manter a consistência lógica. O que acontece com esses nós descartados na

memória do computador, considerando que o Java possui um *Garbage Collector*? Como a ausência de referências (ponteiros) para esses nós influencia esse processo?

Questão 18: Playlist com Reordenação Inteligente (Desafio)

Uma plataforma de música permite que o usuário reordene sua playlist. O desafio técnico consiste em inserir uma música no **meio** da lista, entre dois nós já existentes (A e B). Isso exige a manipulação coordenada de **quatro ponteiros**: os dois do novo nó, o `proximo` do nó A e o `anterior` do nó B.

Requisitos:

- Implemente uma classe `PlaylistDupla` com sentinelas.
- Implemente o método `inserirEntre(String novo, String A, String B)`:
  - Localize os nós A e B e valide se B é o sucessor de A.
  - Realize a "amarração quádrupla"de ponteiros para inserir o novo nó.
- Implemente um método `removerMusica(String nome)` que ajuste os ponteiros dos vizinhos para "fechar o buraco"na lista.
- Na classe `Main`, crie a playlist: [M1, M3]. Insira M2 entre M1 e M3. Remova M1 e verifique se M2 se tornou a nova primeira música (sucessor do `header`).

Pergunta Reflexiva: Ao inserir um nó entre dois nós existentes em uma lista dupla, a ordem das operações de ponteiro é crítica para não "perder"a lista no meio do processo. Descreva, passo a passo, a sequência lógica de atribuições que você utilizou. Por que usar sentinelas (`header/trailer`) torna a implementação de métodos de inserção e remoção muito mais segura e menos propensa a erros de "ponteiro nulo"?

Questão 19: Player de Música em Modo Repetir (Conceito)

Em um player de música, o modo "Repetir Tudo"faz com que, após a última música terminar, a primeira comece a tocar novamente de forma automática. Uma **Lista Circular Simplesmente Encadeada** é a estrutura ideal para esse comportamento, pois o ponteiro `proximo` do último nó não é `null`, mas sim aponta de volta para o primeiro nó. Uma otimização comum (seguindo Goodrich & Tamassia) é manter apenas o ponteiro para o **último nó** (`tail`), pois `tail.proximo` nos dá acesso direto à cabeça da lista.

Requisitos:

- Crie uma classe `No` que armazene o nome da música e o ponteiro para o próximo.
- Crie uma classe `PlayerCircular` que mantenha apenas o atributo `No tail` (inicialmente `null`).
- Implemente os métodos:
  - `void adicionarMusica(String nome)` — insere no final. Lembre-se: se a lista estiver vazia, o nó aponta para si mesmo. Se não, o novo nó aponta para a antiga cabeça, e o `tail` aponta para o novo nó.
  - `void exibirPlaylist()` — percorre a lista e imprime as músicas. **Atenção:** Como não existe `null`, você deve salvar a referência inicial e parar quando o ponteiro "der a volta".
- Na classe `Main`, adicione 3 músicas e exiba a playlist.

Pergunta Reflexiva: Em uma lista encadeada simples tradicional, a condição de parada de um laço `while` é `atual != null`. Em uma lista circular, essa condição causaria um loop infinito. Explique, detalhadamente, qual lógica você utilizou para garantir que o método `exibirPlaylist` percorra todos os elementos e pare exatamente após completar uma volta.

Questão 20: Gerenciador de Turnos de Jogo de Tabuleiro (Prática)

Em jogos de tabuleiro como Banco Imobiliário ou War, os jogadores realizam suas jogadas em rodadas cíclicas. Após o último jogador, o turno volta para o primeiro. Além disso, jogadores podem ser eliminados durante a partida.

Requisitos:

- Crie uma classe `Jogador` com `nome` e `pontuacao`.
- Implemente a classe `GerenciadorTurnos` (Lista Circular) com o método `proximoTurno()`:
  - Este método deve apenas mover um ponteiro interno para o próximo nó e retornar o jogador da vez.
- Implemente o método `eliminarJogador(String nome)`:
  - Localize o jogador e remova-o da lista, ajustando os ponteiros para manter a circularidade.
  - Trate o caso especial onde resta apenas um jogador (o vencedor).
- Na classe `Main`, simule 3 rodadas de um jogo com 4 jogadores e elimine o 2º jogador após a primeira rodada.

Pergunta Reflexiva: No gerenciamento de turnos, a operação de "passar a vez"é extremamente frequente. Qual a complexidade de tempo (O) dessa operação em uma lista circular? Compare essa eficiência com o uso de um `ArrayList`, onde remover o jogador do índice 0 e inseri-lo no final a cada turno exigiria deslocar todos os outros elementos.

Questão 21: Escalonador Round Robin de Processos (Desafio)

Um Sistema Operacional utiliza o algoritmo **Round Robin** para distribuir o tempo de CPU entre vários processos. Cada processo recebe uma fatia de tempo fixa (Quantum). Se o processo não terminar nesse tempo, ele volta para o fim da fila; se terminar, é removido da lista.

Requisitos:

- Crie uma classe `Processo` com `id`, `nome` e `tempoExecucao` (restante).
- Implemente o escalonador em uma `ListaCircular`:
  - O escalonador deve percorrer a lista. A cada nó, subtraia o `quantum` (ex: 20ms) do `tempoExecucao`.
  - Se o tempo chegar a 0, remova o nó da lista circular.
  - Continue o ciclo até que a lista esteja vazia.
- Exiba o estado da fila a cada ciclo de escalonamento.
- Na classe `Main`, insira 3 processos com tempos diferentes (ex: 50ms, 20ms, 30ms) e execute o escalonador com quantum de 20ms.

Pergunta Reflexiva: Durante o escalonamento Round Robin, o "ponteiro atual"está sempre em movimento. Ao remover um processo que acabou de terminar, você deve garantir que a circularidade da lista permaneça intacta. Descreva como sua implementação trata a remoção de um nó em uma lista circular: como você encontra o nó anterior para religar a lista e o que acontece se o nó removido for o `tail`?

Questão 22: Carrossel de Imagens de um Site (Conceito)

Websites de e-commerce e portfólios utilizam carrosséis de imagens onde o usuário pode clicar em "Próximo"ou "Anterior"indefinidamente. Em uma **Lista Circular Duplamente Encadeada**, o primeiro nó (`head`) está conectado ao último (`tail`) em ambos os sentidos: `head.anterior` aponta para o `tail` e `tail.proximo` aponta para o `head`. Isso permite uma rotação infinita para qualquer direção sem encontrar o valor `null`.

Requisitos:

- Crie uma classe `NoDuplo` que armazene o nome da imagem (`String`), e os ponteiros `anterior` e `proximo`.
- Implemente a classe `CarrosselCircular` com apenas o ponteiro `head`.
- Implemente o método `adicionarImagem(String nome)`:
  - Se for a primeira imagem, ela deve apontar para si mesma nos dois sentidos.
  - Se houver outras, insira no final, garantindo que as ligações entre o novo nó, o antigo último e a cabeça sejam atualizadas.
- Implemente `exibirHorario()` e `exibirAntiHorario()` para demonstrar a circularidade bidirecional.
- Na classe `Main`, adicione 3 imagens ("Foto1", "Foto2", "Foto3") e exiba-as nos dois sentidos.

Pergunta Reflexiva: Quando uma lista circular dupla possui apenas um único elemento, para onde apontam seus ponteiros `proximo` e `anterior`? Como essa configuração "reflexiva"evita que o sistema tente acessar uma referência nula?

Questão 23: Player de Música Premium com Loop Infinito (Prática)

Diferente de um player básico, a versão Premium permite que, ao clicar em "Anterior"estando na primeira música, o player salte imediatamente para a última música da playlist, mantendo o fluxo sonoro sem interrupções.

Requisitos:

- Crie uma classe `Musica` com `titulo` e `artista`.
- Implemente a classe `PlayerPremium` (Lista Circular Dupla) com um ponteiro `musicaAtual`.
- Implemente os métodos:
  - `void proxima()` — move o ponteiro para o próximo nó.
  - `void anterior()` — move o ponteiro para o nó anterior.
  - `void adicionar(Musica m)` — adiciona no final da playlist.
- Na classe `Main`, adicione 3 músicas. Execute a sequência: 1x Próxima, 2x Anterior. Exiba a música tocando em cada passo, destacando o momento em que a lista "dá a volta"do início para o fim.

Pergunta Reflexiva: Na implementação da operação `anterior()` em uma lista circular dupla, o custo computacional é $O(1)$. Como seria a complexidade dessa mesma operação se estivéssemos usando uma **Lista Circular Simples** (onde não há o ponteiro `anterior`)? Por que a bidirecionalidade é essencial para sistemas de navegação fluida?

Questão 24: Seletor de Heróis em Jogo de Luta (Desafio)

Em jogos de luta, o seletor de personagens permite navegar entre heróis para a esquerda e direita. Ao escolher um herói (por exemplo, em um modo onde personagens não podem ser repetidos), ele deve ser **removido** da lista de seleção. A remoção em uma lista circular dupla é a operação mais complexa, pois exige religar as pontas caso o herói removido seja o único, o primeiro ou o último da sequência.

Requisitos:

- Crie uma classe `Heroi` com `nome` e `classe` (ex: Guerreiro, Mago).
- Implemente a classe `SeletorHerois` (Lista Circular Dupla) com os métodos:
  - `void adicionar(Heroi h)` — insere no final.
  - `void remover(String nome)` — localiza o herói e o remove. Trate com rigor:
    1. Remoção do único herói da lista (a lista deve ficar `null`).
    2. Remoção de um herói no meio, início ou fim, mantendo sempre a conexão circular

dupla entre os vizinhos.

  - `void exibirOpcoes()` — exibe todos os heróis disponíveis.

- Na classe `Main`, adicione 4 heróis. Simule a remoção do herói da cabeça, de um herói no meio e, por fim, remova todos até que a lista fique vazia. Exiba o estado da lista após cada remoção.

Pergunta Reflexiva: A remoção do **último elemento** exige que `head` seja `null`. O que ocorre se aplicar a lógica padrão (`ant.prox = prox`) em uma lista de um só nó? Discorra sobre o risco de *dangling pointers*.